Nyuwe

System Documentation

The complete, current-state reference for the Nyuwe marketplace platform: architecture, the unified payment engine, the double-entry ledger, every revenue system, storage, deployment, and the operational knowledge that keeps it all honest.

Revision
11 August 2026 — authoritative

Platform
nyuwe-g9vtf.ondigitalocean.app

Source of truth
SYSTEM_DOCUMENTATION.md (repo)

CONTENTS

What's in this document

SECTION 01

Platform overview & stack

Nyuwe (formerly ProQuote / Tonse Hub — the lowercase `tonse` in code, storage keys and event names is deliberate and kept) is a Zambian B2B/B2C marketplace. Buyers post **inquiries**; sellers and service providers answer with **quotes**; a verified payment funds **escrow**; goods change hands against a **collection code**. Around that core sit a labour job board, event ticketing, shop subscriptions, seller advertising (banners plus a Spotlight pop-up), financing/loans, and a full double-entry ledger with real mobile-money and card payments via DPO.

LAYER	TECHNOLOGY
Frontend	React 18 + Vite + TypeScript, Tailwind, motion/react, React Router, PWA with a hand-rolled service worker (`public/sw.js`)
Backend	NestJS + TypeORM on PostgreSQL 15/16
Payments	DPO (Direct Pay Online) v6 XML API, plus an in-process sandbox provider for development
Storage	Local filesystem *or* DigitalOcean Spaces (S3-compatible), behind one driver interface
Hosting	DigitalOcean App Platform (primary) + a legacy Render deployment

Repository layout

```
/                    Vite frontend (src/), all *.md documentation
/src/pages           Route-level pages (dashboards, discover, tickets, payment return…)
/src/components      UI — schema-driven shells + feature views
/src/services        API clients, category catalog, event-bus helpers
/backend/src/modules  One NestJS module per domain (37 modules)
/backend/src/database/migrations   TypeORM migrations (prod runs these on boot)
/docker-compose.yml   Local Postgres (+ optional pgAdmin / backend containers)
```

> **Response envelope.** Every backend success is wrapped `{statusCode, message, data}` by a global interceptor; errors are raw Nest exception bodies. The frontend `apiClient` unwraps `.data` automatically — any script driving the API directly must unwrap it itself.

SECTION 02

Frontend architecture

Schema-driven UI

Dashboards are **generated, not hand-written**: `MasterAccountSchema` (per role) plus archetype detection (per category) decide which tabs and views each user gets. To add or move a navigation item you edit the schema files, not JSX. `DynamicAccountRenderer` renders the views those schemas name — quote details, order details, venture account, ads manager, job posts, and the rest.

The event bus

Cross-cutting signals are window CustomEvents named `tonse:<kebab>`, each with a small helper module exporting the event name and a `subscribeToX()` that returns its unsubscriber. In use: `tonse:data-revalidated`, `tonse:quotes-changed`, `tonse:inquiries-changed`, `tonse:schedule-date-selected`, `tonse:ad-inquiry-intent`, `tonse:shopping-intent` (the Spotlight trigger), `tonse:hosted-payment`, and more.

Global widgets & payment kit

Always-mounted, zero-prop, self-gating components sit beside the router in `App.tsx`: `OfflineBanner`, `InstallAppBanner`, `FloatingHub` (minimize-to-bubble background mode) and `SpotlightAd`. The payment kit is shared everywhere: `PaymentSheet` (the one payment modal — amount, phone, MTN/Airtel/Zamtel, card), `PushPaymentWait` (approve-on-phone polling card), `beginHostedPayment()` (redirect hand-off to DPO's hosted page) and `PaymentReturnPage` at `/payment/return` — a public route, because guest ticket buyers land there too.

Offline / PWA

`public/sw.js` (plain JS on purpose) warms the app shell and images from `asset-manifest.json`; the API client's SWR option revalidates in the background and fires `tonse:data-revalidated`. After your *own* mutation, refetch with network forced — the SWR cache may still hold the pre-mutation body.

> **Android rendering rule.** Never use translucent border utilities (`border-…/NN`) on rounded cards — Chrome on Mali GPUs smears and ghosts them on real phones. Opaque hex borders only.

SECTION 03

Backend modules & identity

One NestJS module per domain, 37 in all:

`admin` `ads` `audit` `auth` `billing` `calendar-events` `care-plans` `categories` `collection` `consents` `files` `financing` `idempotency` `identity-audit` `inquiries` `job-board` `jobs` `ledger` `loans` `notifications` `orders` `payments` `portfolio` `products` `quotes` `referrals` `reports` `reviews` `schedules` (dead — use calendar-events) `shops` `site-settings` `storage` `storefront` `team` `tickets` `users` `venture`

Roles & identity

- **Five live roles:** `BUYER`, `SELLER`, `SERVICE_PROVIDER`, `ADMIN`, `PROMOTER`.
- A company can hold up to three profile rows on one `users` row and switch the active one (header → Role Manager); one admin approval covers all its sides.
- Display IDs: inquiries get QIDs (the charset deliberately skips 6, 7, 8 and 9); users get USER-ids. `IDENTITY_AND_DISPLAY_IDS.md` remains the authority for that subsystem.
- Sub-admins are deny-by-default (`AdminPermissionsGuard`); undecorated admin routes are primary-admin-only.

SECTION 04

The marketplace core

1. **Inquiry** (buyer). Category-driven dynamic form (json `attributes`). `EXPRESS` (pay on quote) vs `STANDARD` (PO-style). Targeted inquiries go to one shop; landing-page guests can compose one that defers submission until after login.
2. **Quote** (seller/provider). Priced answer. Status path: `PENDING → ACCEPTED → PAYMENT_PENDING → PAID → PENDING_COLLECTION / AWAITING_PICKUP → COMPLETED / HANDED_OVER`, with terminal exits `CANCELLED / REFUNDED`. Money statuses are **server-written only**.
3. **Payment = escrow.** `POST /payments/checkout {quoteId}` reads the price server-side and starts a PSP collection. On *verified* success, `fundEscrow()` — in one transaction — posts the `ESCROW_FUNDED` journal, creates the **Order**, flips the quote `PAID`, closes the inquiry and mints the **collection code** (`PQ-XXXXXXXX`).
4. **Collection.** The handover flow looks the quote up *by* collection code; completion releases escrow to the seller's venture balance, net of commission, after `PAYOUT_HOLD_HOURS`.

> **Closed bypass (Aug 2026).** `POST /orders` used to flip any quote PAID with zero payment, and three separate UIs called it. It now rejects priced non-LOAN quotes — paid orders exist only through the payment flow. Direct product purchases use `direct-order.service.ts` (born-PAID quote + order + escrow journal in one transaction).

Lifecycle views never duplicate an item across tabs — the `lifecycleFilters` helpers put every item in exactly one stage bucket.

SECTION 05

The payment engine

Provider abstraction

`PAYMENT_PROVIDER=sandbox|dpo` picks the adapter at boot, behind one interface. The **sandbox** makes no network calls: collections park as pending and every flow has an in-app *Simulate approval* button (each simulate endpoint refuses when the live provider is configured). The **DPO adapter** is real money: `createToken` always reserves the transaction and hosted-page URL; for **mobile money with a phone number** it then calls `GetMobilePaymentOptions` + `ChargeTokenMobile` to push the charge to the payer's handset — approved **inside our UI**, no redirect (MTN and Airtel Zambia confirmed; any miss falls back to the hosted page). Cards always use DPO's hosted page, keeping PCI out of scope.

> **Trust model.** DPO's Payment Notification is unsigned, so it is only a *hint*. Every settlement re-verifies with `verifyToken` first, and an amount mismatch refuses to settle. Result codes: 000 paid · 901/903/904 dead · 801–804/902/950 are *our request's* errors (503, never "payment failed").

One engine, every pay point

Every initiator writes a `psp_transactions` row (`context.kind` is the discriminator) and returns the same shape (`reference, provider, status, redirectUrl?, instruction?`). Settlement arrives via webhook (`POST /webhooks/psp?secret=…`), the payer's verify (`POST /payments/checkout/:ref/verify`) or sandbox simulate — all idempotent, all re-verified. `verifyAndSettle` dispatches to a `fund*` step: FOR-UPDATE lock, `SUCCESSFUL` short-circuit, ledger journal with a `<kind>:tx.id` idempotency key, then the entity flip.

PAY POINT	KIND / ROUTE	ON VERIFIED SUCCESS
Quote / escrow	quoteId set · `POST /payments/checkout`	ESCROW_FUNDED + Order + collection code
Venture deposit	default · venture deposit route	Seller balance credit
Ad purchase	`AD_PURCHASE` · `/ads/:id/checkout`	AD_REVENUE; ad → review queue
Job-post fee	`JOB_POST_FEE` · `/job-postings/:id/checkout`	JOB_BOARD_REVENUE; post → review queue
Ticket sale (guest)	`TICKET_SALE` · `/tickets/public/checkout/:ref/pay`	Commission + seller net; tickets minted
Subscription	`SUBSCRIPTION_FEE` · `/billing/subscription/checkout`	SUBSCRIPTION_REVENUE; paidUntil +30 d
Loan disbursement	LOAN context (a collection *from the lender*)	Escrow for the financed order

Guest ticket payments carry `counterpartyId = NULL`; ownership is the reference plus the kind check, exposed only through unauthenticated routes on the public tickets controller. Reference prefixes

(`CHK/ADV/JPF/TPF/SUB/DEP`) let the return page pick the right verify endpoint. Ads and job fees can alternatively be paid from the seller's venture balance (a pure ledger transfer, no PSP round-trip).

> **Rule: never add a new instant-success payment path.** Copy the `initiateJobPostFee` / `fundJobPostFee` pair.

SECTION 06

Ledger & money

Double-entry and append-only. Database triggers enforce balanced journals; every position (escrow, venture balances) is a derived view, never a stored flag. The legacy `payments` table is a frozen history — the ledger is authoritative.

Chart of accounts (11, all ZMW)

ACCOUNT	MEANING
PSP_HOLDING	Our claim on money sitting at the PSP — must reconcile to DPO's balance
ESCROW_LIABILITY	Money owed on unresolved deals (per-quote positions)
SELLER_PAYABLE	The venture balances — earned, not yet paid out
REFUND_PAYABLE / PSP_PAYOUT_IN_FLIGHT	Refunds owed · payouts instructed but unconfirmed
PLATFORM_COMMISSION_REVENUE	Commission on escrow releases and ticket sales
AD_REVENUE · JOB_BOARD_REVENUE · SUBSCRIPTION_REVENUE	The other three platform revenue streams
PSP_FEE_EXPENSE · SUSPENSE	Fees we truly bear · unattributed receipts (an alarm, not a resting place)

Platform earnings — the admin's own account: `GET /admin/platform-earnings` sums the four revenue accounts live from ledger entries; the Admin → Accounts tab leads with the Platform Earnings card (total + per-stream). Journal types are a DB enum (`ESCROW_FUNDED/RELEASED, VENTURE_DEPOSIT, AD_PURCHASE, AD_REJECTED_REFUND, JOB_POST_FEE, SUBSCRIPTION_FEE, TICKET_SALE, REFUND_*, PAYOUT_*, REVERSAL, ADJUSTMENT, OPENING_BALANCE`); adding one needs an `ALTER TYPE … ADD VALUE` migration. Accounts seed insert-only at boot — adding one is just an array entry.

SECTION 07

Advertising

Seller-purchased, admin-priced, admin-approved. Lifecycle `PENDING_PAYMENT → PENDING_APPROVAL → APPROVED/REJECTED` (+ derived `EXPIRED`). Approval slides the paid window forward if review was slow — a queue must never cost paid days. Rejection refunds the full price to the seller's venture balance. A boot-time sweep deletes any ad whose media object is missing from storage (with an abort valve if storage lists zero objects).

Product 1 — on-page placements

`HOMEPAGE_CENTER`, `SECONDARY_SIDEBAR`, `CATEGORY_SIDEBAR` (category-targetable). One shared `baseRatePerDay` — price scales with days, never with how many placements are ticked — plus duration discount tiers. Rendered by `AdCarousel` (10-second rotation, 60-second fetch cache) via `AdRail` and `InlineAdSlot`.

Product 2 — the Spotlight pop-up

A premium modal advert shown **over** the screen at moments of shopping intent (`tonse:shopping-intent`: a buyer picks a category, or a Discover visitor filters one). Booked **exclusively** (never mixed with on-page placements — one ad row carries one price) at its own `popupRatePerDay`.

- **Rationing is server-enforced:** every hand-out writes an `ad_popup_impressions` row per viewer (account id, or an anonymous browser key for guests); the cap is `popupMaxPerSession` per `popupMinMinutesBetween` window. Clearing the browser cannot buy more.
- **Fair round-robin:** each viewer sees the ad they've seen least recently, tie-broken toward the advertiser with the fewest total impressions — a new advertiser catches up; nobody dominates. Category-targeted ads win relevance.
- **Kill switch:** `popupEnabled` off stops new bookings *and* all serving. Impressions prune at 30 days; clicks are stamped for engagement stats.
- **Respectful by construction:** 1.4 s delay after the trigger, never shown to sellers/providers/admins, never on payment or auth routes, z-index below every real modal.

Click-through is one shared implementation for all ad surfaces: a buyer lands in the inquiry funnel pre-targeted at the advertiser (attribution riding as `?ad=` / stashed intent), a guest goes via login, a non-buyer to the shop page.

SECTION 08

Job board, ticketing & billing

Labour job board

Any account may post; postings are labour-trades-only and admin-moderated (`PENDING_PAYMENT? → PENDING_APPROVAL → APPROVED → FILLED/CLOSED`; `REJECTED` → edit-and-resubmit without paying again). The admin-controlled posting fee lives in billing settings, snapshotted onto each posting. The feed is the whole board; applying self-gates on holding an employment account. Applications require an **Application Letter** plus any poster-demanded documents (encrypted secure uploads); contact details reveal only on ACCEPT — in both directions. Machinery-hire stayed on the inquiry funnel.

Event ticketing

Events-family sellers create events with priced tiers; guests buy through public share links (`/e/EVT-XXXXXX`) with no account. Two steps: park a PENDING order priced server-side, then a real PSP payment. On verified payment, one locked transaction re-checks stock, mints per-attendee QR ticket codes, and posts the `TICKET_SALE` journal — gross to PSP holding, seller's net to their venture balance, commission to the platform. Tickets are emailed best-effort; check-in runs through scanner roles.

Billing switches (`billing_settings`, one row)

KNOB	EFFECT
`subscriptionsEnabled`	Master switch: ON → buyers pay tiered quotation fees AND shops need an active monthly subscription (full-screen paywall otherwise)
`quoteTiers` / `targetedInquiryFee`	Buyer-side quotation fees
`monthlyFee`	Shop subscription — real payment; paidUntil extends +30 days from max(now, paidUntil)
`jobPostingFeeEnabled` / `jobPostingFee`	Job board posting fee

Ad pricing and Spotlight knobs live in `ad_settings`; ticket commission in `event_ticket_settings`. Every knob fails toward its safe direction: a fetch failure never waives fees platform-wide, and never falsely paywalls a paying shop.

SECTION 09

Files, storage & notifications

Storage drivers

One `StorageDriver` interface, chosen by `STORAGE_DRIVER=filesystem|spaces`. On **Spaces** (production): public uploads under `uploads/` are world-readable and served from the CDN; sensitive objects under `secure-uploads/` carry **no public ACL**. On **filesystem** (local / legacy Render): served from `/uploads` — and a production boot with this driver and no mounted `UPLOADS_DIR` logs a screaming ERROR, because a container disk is wiped on every restart (the trap that once destroyed uploads).

Sensitive categories (KYC, payslips, job-application documents) are **AES-256-GCM encrypted above the driver** and served only through the authenticated `GET /files/secure/:file`. On the frontend, never `<img>` a secure URL — use `SecureFile` (PDF-aware), branching per-URL with `isSecureFileUrl`.

Notifications

Typed in-app notifications per domain plus real Web Push (VAPID) through `notifyUsers()`. Adding a notification type touches up to four places — enum, dispatch, frontend rendering, route mapping. `FloatingHub` keeps a minimized session alive as a floating bubble.

SECTION 10

Admin panel

- **Overview** — platform stats.
- **Users** — User Manager, sub-admins (deny-by-default permission codes), verification queue, suspend/verify/reject.
- **Inquiries / Quotes / Transactions** — moderation and the frozen payments history.
- **Ledger** — chart of accounts led by the **Platform Earnings** card, trial balance, journal browser, escrow positions.
- **Ads** — base daily rate + discount tiers, the Spotlight card (kill switch, rate, rationing knobs), and the approval queue (Spotlight bookings badged).
- **Job board** — posting review queue.
- **Subscriptions** — the monetization master switch and all fees.
- **Tickets** — commission percentage.
- **Site settings** — landing-page switch; **Reports** — user moderation; **Audit log** — every admin action recorded.

SECTION 11

Deployment & environments

Primary — DigitalOcean App Platform (region `lon`): a static site (Vite `dist/`), the `api` Docker service (`backend/Dockerfile`), a dev Postgres, and the Spaces bucket `nyuwe-media` with CDN. Ingress: `/api` → api, `/` → web. Deploys run automatically on every push to `walandadev-tech/nyuwe` `main`. **Production applies TypeORM migrations on boot** (`DB_RUN_MIGRATIONS=true`, synchronize off) — a failed migration fails the deploy, which is the safety. Dev uses synchronize.

Legacy — Render (`tonse-web.onrender.com`): still running an older blueprint; do not verify new work against it. **Git remotes:** push `main` to all three — `walanda` (deploys), `tonse`, `github`.

Environment variables that matter

VARIABLE(S)	NOTES
`DB_*`, `DB_SSL=no-verify`	Postgres; SSL flag needed on DO
`JWT_EXPIRATION=1h` · `JWT_REFRESH_EXPIRATION=7d`	**Always with a unit** — a bare `3600` parses as milliseconds and logs everyone out in seconds
`PII_ENCRYPTION_KEY`	Boot fails closed in production without it
`PAYMENT_PROVIDER` + `DPO_COMPANY_TOKEN` / `DPO_DEFAULT_SERVICE_TYPE` / `DPO_NOTIFICATION_SECRET` / `DPO_REDIRECT_URL` / `DPO_BACK_URL`	Live payments; currently on DPO **test** credentials — swap the company token to go live
`STORAGE_DRIVER` + `SPACES_*`	Object storage (bucket, keys, CDN base, prefixes)
`PLATFORM_COMMISSION_PERCENT`, `PAYOUT_HOLD_HOURS`, `WEB_PUSH_*`, `ADMIN_*`, `PROMOTER_INVITE_KEY`, `CORS_ORIGINS`, SMTP	Commission, payout hold, push keys, admin seeder, invite key, CORS, ticket email

> **Two rules for .env values:** unquoted, and no trailing `# comment` — dotenv truncates at `#`. **DPO portal:** set the Payment Notification URL to `https://<api-host>/api/webhooks/psp?secret=<DPO_NOTIFICATION_SECRET>`.

SECTION 12

Local development & verification

```
# Postgres (root .env needs DB_PASSWORD, PII_ENCRYPTION_KEY for compose)
docker compose up -d postgres
# Backend — scratch port if 3001 is taken; dev uses synchronize
cd backend && PORT=3099 npm run start:dev
# Frontend
npm run dev
```

- Type checks: `npx tsc --noEmit` in both roots; bundle proof: `npm run build`.
- Auth routes are throttled at 5/minute — scripts that register and log in repeatedly will 429; wait out the window.
- **E2E convention:** standalone node scripts drive the HTTP API on the scratch port with the sandbox provider and simulate endpoints, and double-check persisted state directly in Postgres as an independent oracle. The payments overhaul shipped on a 30/30 suite; the Spotlight system on 13/13.
- The boot media sweep deletes ads whose media isn't in storage — test fixtures need real uploads, or clean up after.
- No browser automation in the default environment: UI wiring is proven by tsc + vite build; visual checks are manual (check Android for the border rule).

SECTION 13

Known gotchas — don't relearn these

1. **JWT lifetimes need units** (`1h` , never `3600` — bare numbers are milliseconds).
2. **dotenv truncates at** `#` — no inline comments, no quotes on values.
3. **json columns are filtered in JS after load, never in SQL** (`placements` , `attributes` …), and json-DISTINCT 500s — Postgres has no equality operator for json.
4. **pg returns decimals as strings** — `Number()` -coerce before arithmetic reaches the UI.
5. **DB enum values** need `ALTER TYPE … ADD VALUE IF NOT EXISTS` migrations and can never be dropped; new migration timestamps must exceed the latest.
6. **Nested DTOs** need `@ValidateNested + @Type` classes, or the global whitelist pipe silently blanks them.
7. **Money statuses are server-written only**; every funding step is idempotent; webhooks are hints — always re-verify with the provider before moving a ngwee.
8. **Opaque borders on rounded cards** — translucent strokes smear on Android Mali GPUs.
9. **Parallel sessions share this checkout** — re-check `git status` before editing shared files; someone else may have pushed.
10. **Old docs lie** — the repo's completion-summary files predate most of today's systems. `SYSTEM_DOCUMENTATION.md` wins every disagreement.
